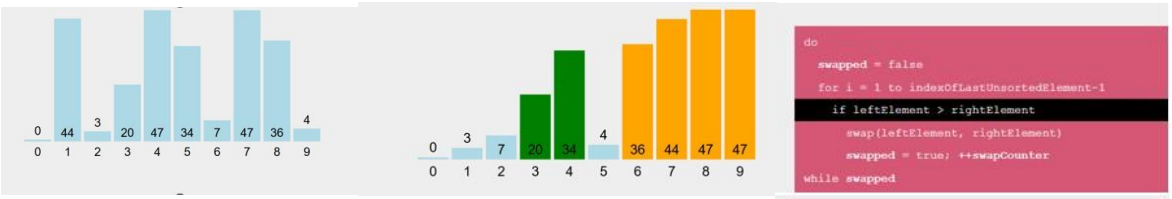
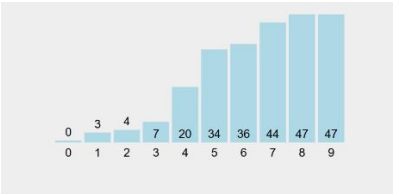
BUBBLE SORT

El algoritmo Bubble Sort funciona comparando elementos adyacentes e intercambiándolos si están en el orden incorrecto. En cada iteración, el elemento más grande se desplaza progresivamente hacia el final del arreglo, fenómeno conocido como “burbujeo”. Este proceso se repite hasta que no se requieren más intercambios.

Proceso BUBBLE SORT



En el ejemplo, se observa que el algoritmo requiere múltiples pasadas sobre el arreglo para lograr el ordenamiento completo. A medida que aumenta el tamaño del arreglo, el número de comparaciones crece rápidamente, lo que confirma su comportamiento cuadrático



Al terminar, el arreglo queda ordenado porque los elementos mayores se fueron moviendo al final en cada iteración, hasta que ya no hubo cambios.

Relación con Big-O

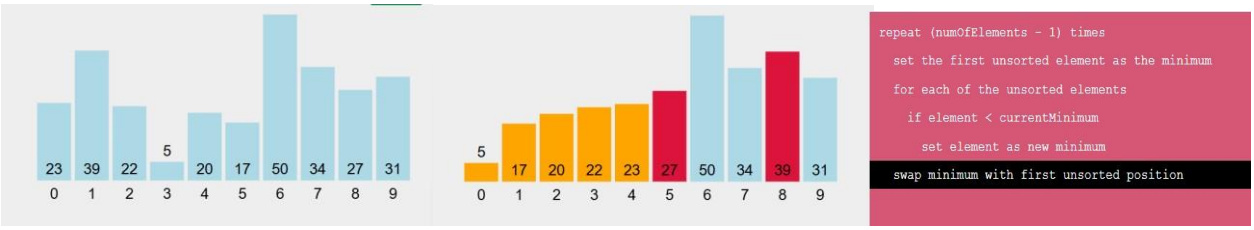
En el peor caso se hacen comparaciones:

$(n-1) + (n-2) + ... + 1 \quad n(n-1)/2$

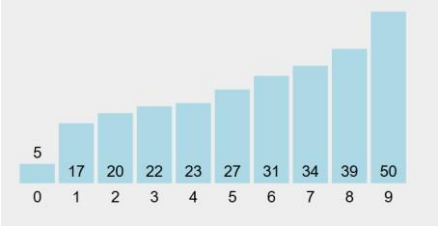
Complejidad: $O(n^2)$

SELECTION SORT

El algoritmo Selection Sort funciona seleccionando el elemento más pequeño del arreglo en cada iteración y colocándolo en su posición correcta. A diferencia de otros algoritmos, no realiza múltiples intercambios, sino que hace una sola selección por ciclo, asegurando que cada posición quede ordenada progresivamente.



Aunque Selection Sort reduce la cantidad de intercambios en comparación con Bubble Sort, sigue realizando el mismo número de comparaciones, lo que provoca un crecimiento cuadrático. Por esta razón, no es eficiente para arreglos grandes.



El arreglo queda ordenado porque en cada paso se coloca el elemento más pequeño en su posición correcta.

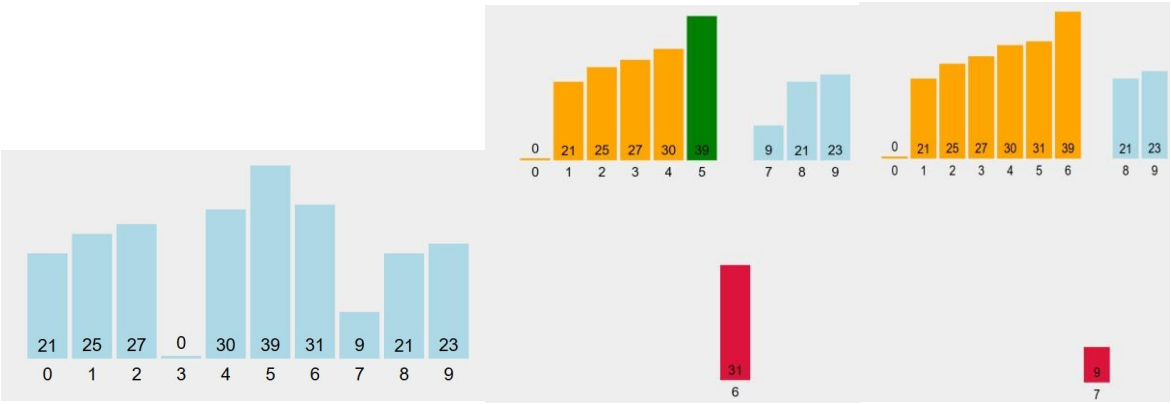
Análisis Big-O

Número de comparaciones: n(n-1)/2

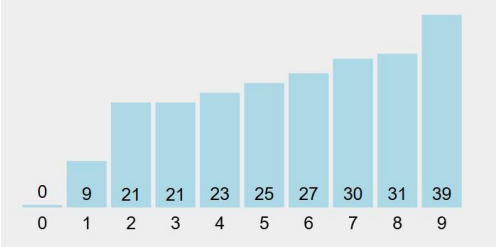
Complejidad: **$O(n^2)$**

INSERTION SORT

El algoritmo Insertion Sort ordena los elementos construyendo una lista ordenada de manera progresiva. En cada paso, toma un elemento y lo inserta en la posición correcta dentro de la parte ya ordenada del arreglo. Este proceso continúa hasta que todos los elementos han sido colocados en su posición adecuad



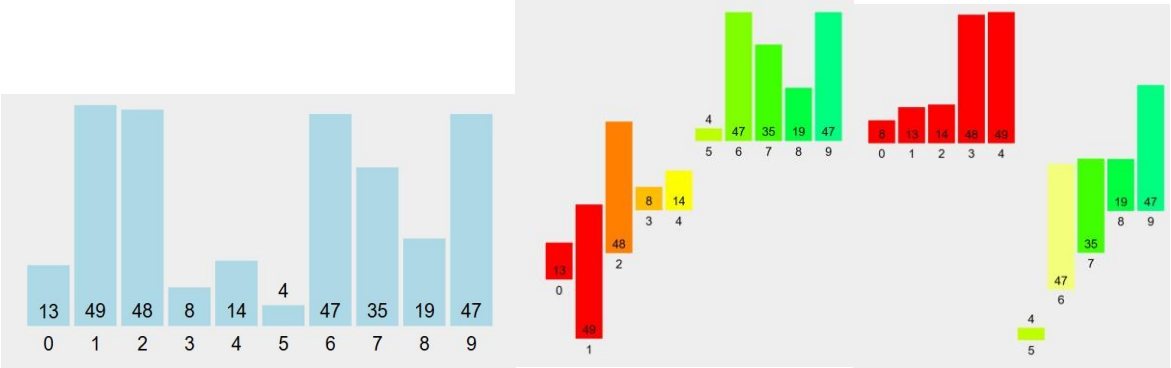
Insertion Sort es más eficiente que otros algoritmos cuadráticos cuando los datos están parcialmente ordenados, ya que reduce el número de comparaciones. Sin embargo, en arreglos grandes o desordenados, su rendimiento sigue siendo limitado debido a su crecimiento cuadrático.



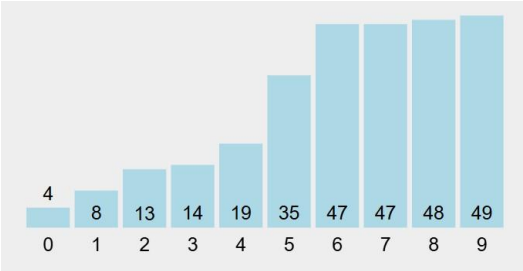
Al finalizar el proceso, el arreglo queda completamente ordenado, ya que cada elemento fue insertado en su posición correcta dentro de la lista. Esto demuestra que el algoritmo funciona de manera progresiva, construyendo una solución ordenada paso a paso hasta completar el ordenamiento total

MERGE SORT

El algoritmo Merge Sort funciona. Primero divide el arreglo en partes más pequeñas hasta llegar a elementos individuales. Posteriormente, combina estas partes de manera ordenada, comparando elementos y formando subarreglos ordenados hasta reconstruir completamente el arreglo original.



A diferencia de los algoritmos cuadráticos, Merge Sort mantiene un crecimiento mucho más controlado, lo que lo hace altamente eficiente para grandes volúmenes de datos. Su comportamiento es estable y predecible, independientemente del orden inicial del arreglo.



Al finalizar el proceso, el arreglo queda completamente ordenado debido a la combinación progresiva de subarreglos ordenados. Este método garantiza un ordenamiento eficiente al dividir el problema en partes pequeñas y resolverlas de manera estructurada.

Analisis BIG-O

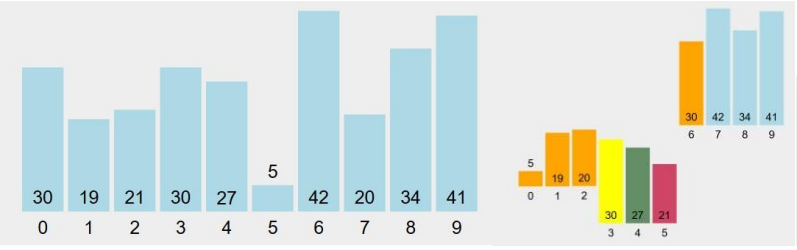
División: log(n)

Combinación: n

Resultado: O(n log n)

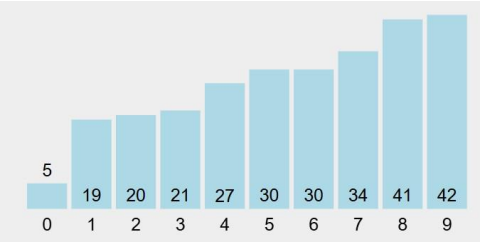
QUICK SORT

El algoritmo Quick Sort ordena los elementos seleccionando un pivote y dividiendo el arreglo en dos subarreglos: uno con los elementos menores al pivote y otro con los mayores. Este proceso se aplica de forma recursiva hasta que todos los elementos quedan ordenados.




```
for each (unsorted) partition
set first element as pivot
  storeIndex = pivotIndex+1
  for i = pivotIndex+1 to rightmostIndex
    if ((a[i] < a[pivot]) or (equal but 50% lucky))
      swap(i, storeIndex); ++storeIndex
  swap(pivot, storeIndex-1)
```

Quick Sort es uno de los algoritmos más eficientes en la práctica debido a su rapidez al dividir el problema en partes más pequeñas. Sin embargo, su rendimiento depende de la elección del pivote, ya que una mala selección puede aumentar el tiempo de ejecución.



Al finalizar el proceso, el arreglo queda completamente ordenado gracias a la división recursiva y la correcta ubicación de los elementos alrededor del pivote. Este método permite un ordenamiento eficiente y rápido en la mayoría de los casos.

ANÁLISIS BIG-O Promedio:

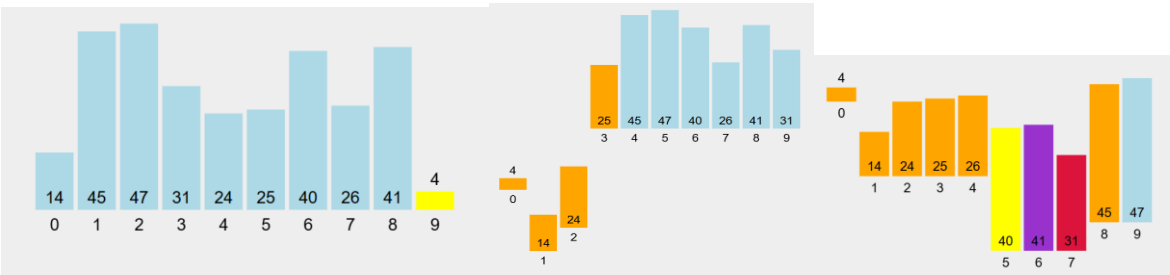
O(n log n)

Peor caso:

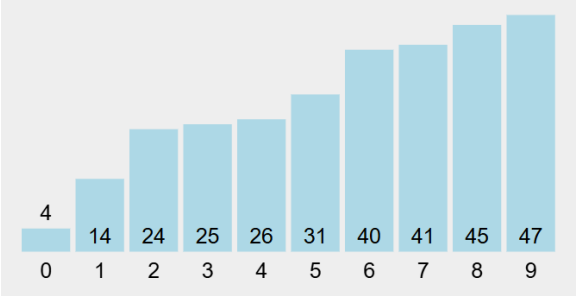
$O(n^2)$

RANDOM QUICK SORT

Random Quick Sort es una variante del algoritmo Quick Sort que selecciona el pivote de forma aleatoria. Esto permite evitar los peores casos provocados por una mala elección del pivote, mejorando el rendimiento promedio del algoritmo.



Al elegir el pivote de manera aleatoria, se reduce significativamente la probabilidad de obtener particiones desbalanceadas, lo que hace que el algoritmo sea más eficiente y estable en la práctica en comparación con Quick Sort tradicional.



Al finalizar el proceso, el arreglo queda completamente ordenado gracias a la división recursiva basada en un pivote aleatorio. Mejora el rendimiento general del algoritmo al evitar casos desfavorables, garantizando un comportamiento más eficiente en la mayoría de las ejecuciones.

Analisis BIG-O

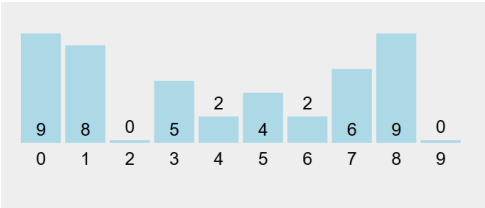
O(n log n)

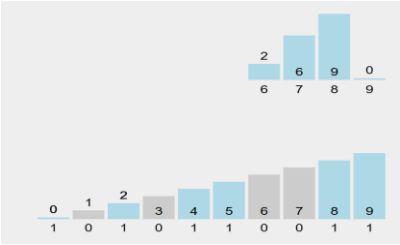
Peor caso:

O(n^2) (muy poco probable)

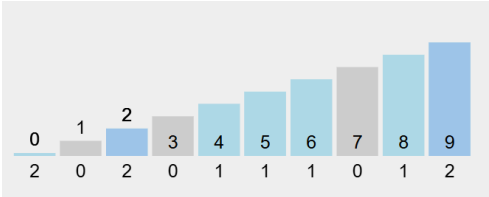
Counting Sort

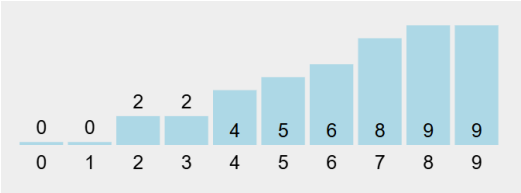
El algoritmo Counting Sort ordena los elementos contando la cantidad de veces que aparece cada valor en el arreglo. Posteriormente, reconstruye el arreglo ordenado utilizando estas frecuencias, evitando comparaciones entre elementos.





Counting Sort es muy eficiente cuando el rango de valores (k) no es mucho mayor que el número de elementos (n). Sin embargo, puede volverse ineficiente en términos de memoria si los valores son muy grandes.





Al finalizar el proceso, el arreglo queda completamente ordenado gracias al conteo de frecuencias de cada elemento. Este método permite un ordenamiento rápido al evitar comparaciones, siendo especialmente útil cuando se trabaja con valores enteros dentro de un rango limitado.

Analisis BIG-O

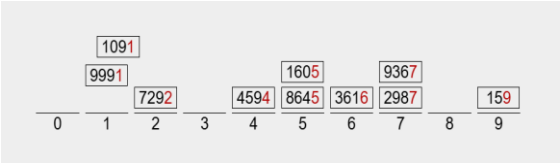
O(n + k)

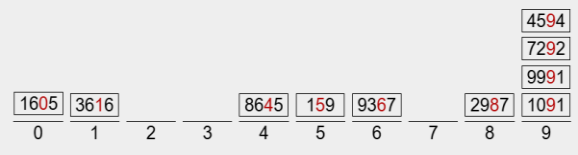
Donde:

- n = número de elementos
- k = rango de valores

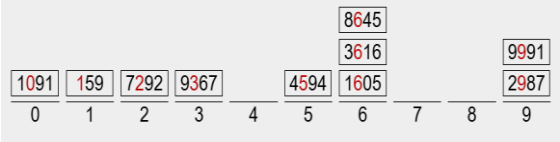
Radix sort

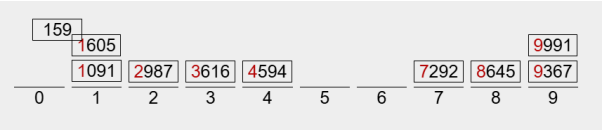
El algoritmo Radix Sort ordena los números procesando cada uno de sus dígitos, desde el menos significativo hasta el más significativo. Para cada dígito utiliza un algoritmo de ordenamiento estable, generalmente Counting Sort, lo que permite mantener el orden relativo de los elementos.





Radix Sort es altamente eficiente cuando los números tienen pocos dígitos, ya que evita comparaciones directas entre elementos. Su rendimiento es estable y puede ser superior a algoritmos O(n log n) en casos específicos.





Al finalizar el proceso, el arreglo queda completamente ordenado gracias a la organización progresiva de los elementos según sus dígitos. Este enfoque permite ordenar de manera eficiente sin necesidad de comparaciones directas, aprovechando la estructura numérica de los datos.

Analisis BIG-O

O(n * k)

Donde:

- n = número de elementos
- k = número de dígitos

Bubble Sort es el más sencillo, pero también el más lento, ya que realiza muchos intercambios innecesarios.

Selection Sort mejora un poco al reducir intercambios, pero sigue siendo lento porque compara todos los elementos.

Insertion Sort puede ser rápido si los datos ya están casi ordenados, lo que lo hace útil en algunos casos específicos.

Merge Sort siempre mantiene un buen rendimiento y es muy confiable, aunque necesita más memoria.

Quick Sort suele ser el más rápido en la práctica, pero depende de cómo se elija el pivote.

Random Quick Sort mejora esto al usar un pivote aleatorio, evitando malos casos.

Counting Sort es diferente porque no compara elementos, sino que cuenta cuántas veces aparece cada número, lo que lo hace muy rápido si el rango de valores es pequeño.

Radix Sort también es especial, ya que ordena los números por dígitos, siendo muy eficiente cuando los números no son muy grandes.

Cuando el tamaño de los datos crece, las diferencias entre los algoritmos se vuelven más claras. Los algoritmos con complejidad $O(n^2)$ empiezan a volverse muy lentos, mientras que los que tienen complejidad $O(n \log n)$ siguen funcionando de manera eficiente. Además, en ciertos casos específicos, algoritmos como Counting Sort y Radix Sort pueden ser incluso más rápidos que los anteriores, siempre y cuando se cumplan ciertas condiciones en los datos.

En conclusión, no existe un único algoritmo que sea el mejor en todos los casos. Sin embargo, para la mayoría de las situaciones, Quick Sort, Random Quick Sort y Merge Sort son las mejores opciones debido a su buen rendimiento. Por otro lado, Counting Sort y Radix Sort pueden ser aún más eficientes en escenarios específicos, mientras que Bubble, Selection e Insertion Sort son más útiles para fines educativos o para trabajar con arreglos pequeños.